

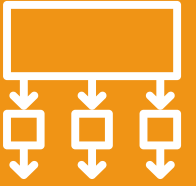


Final Project

Sajjad Ranjbar

ML, winter semester 1403-1404

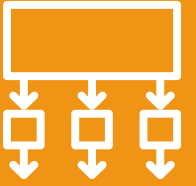
Repository



<https://github.com/phantomf-a-htiw-emag-nuf-A/4321seivom-fo-tesatad-eguh>



EDA





Cleaning

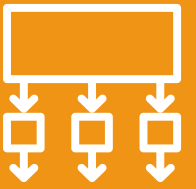
- Convert data frames to numeric
- Drop Null rows
- Join ratings → links → metadata

Save plots

Save csv

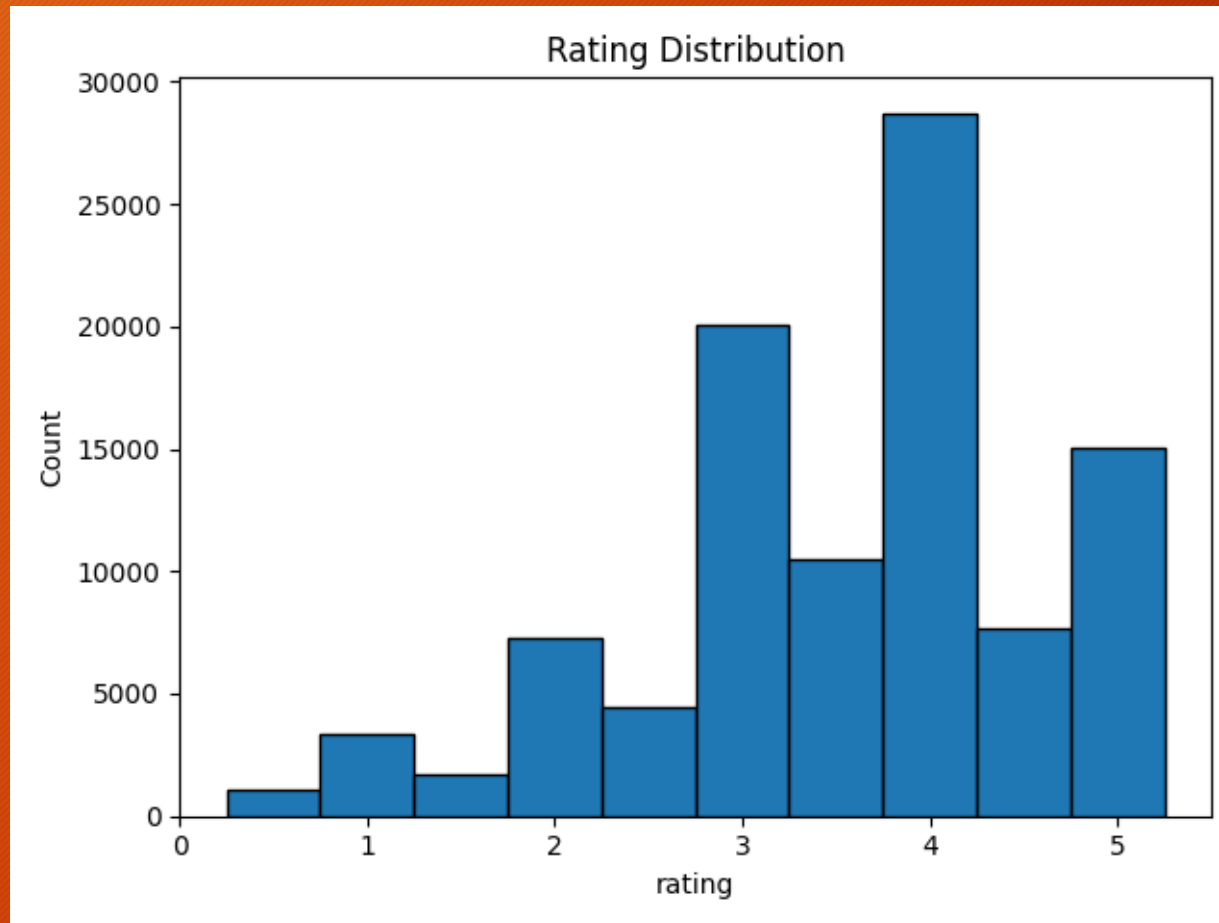
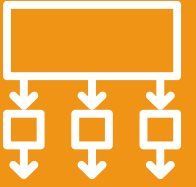
- eda_rowcount_audit
- eda_user_activity_counts
- eda_movie_popularity_counts
- eda_genre_counts
- eda_language_counts
- eda_country_counts
- eda_temporal_monthly

EDA

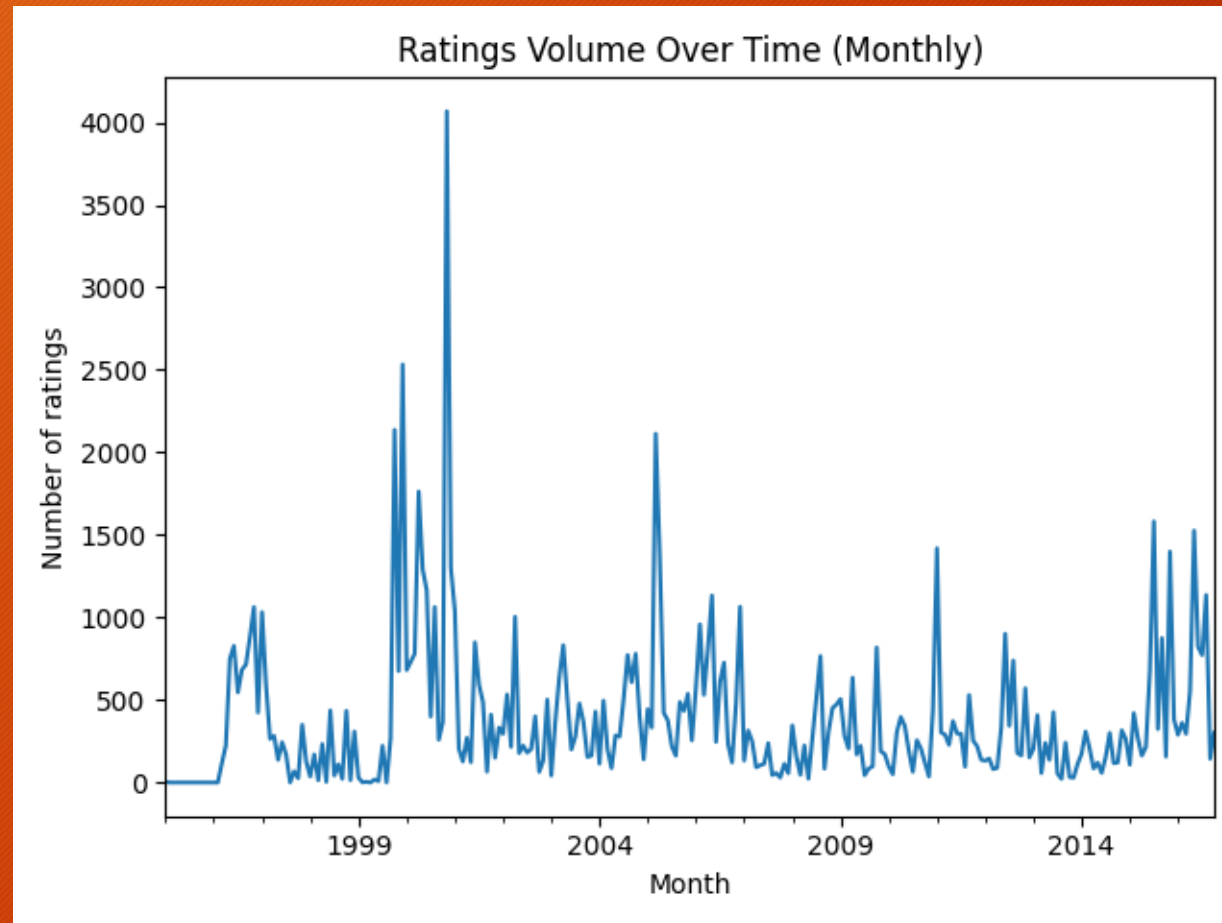
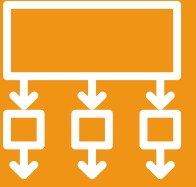


```
{'step': 'load_raw', 'ratings_rows': 100004, 'links_rows': 45843, 'meta_rows': 45466}  
{'step': 'clean_ids', 'links_clean_rows': 45624, 'meta_clean_rows': 45463}  
{'step': 'ratings_join_links', 'ratings_before': 100004, 'r_links_rows': 99898}  
{'step': 'join_with_metadata', 'r_links_before': 99898, 'r_full_rows': 99850}  
{'step': 'filter_valid_rows', 'r_full_rows': 99850, 'unique_users': 671, 'unique_movies': 9025}
```

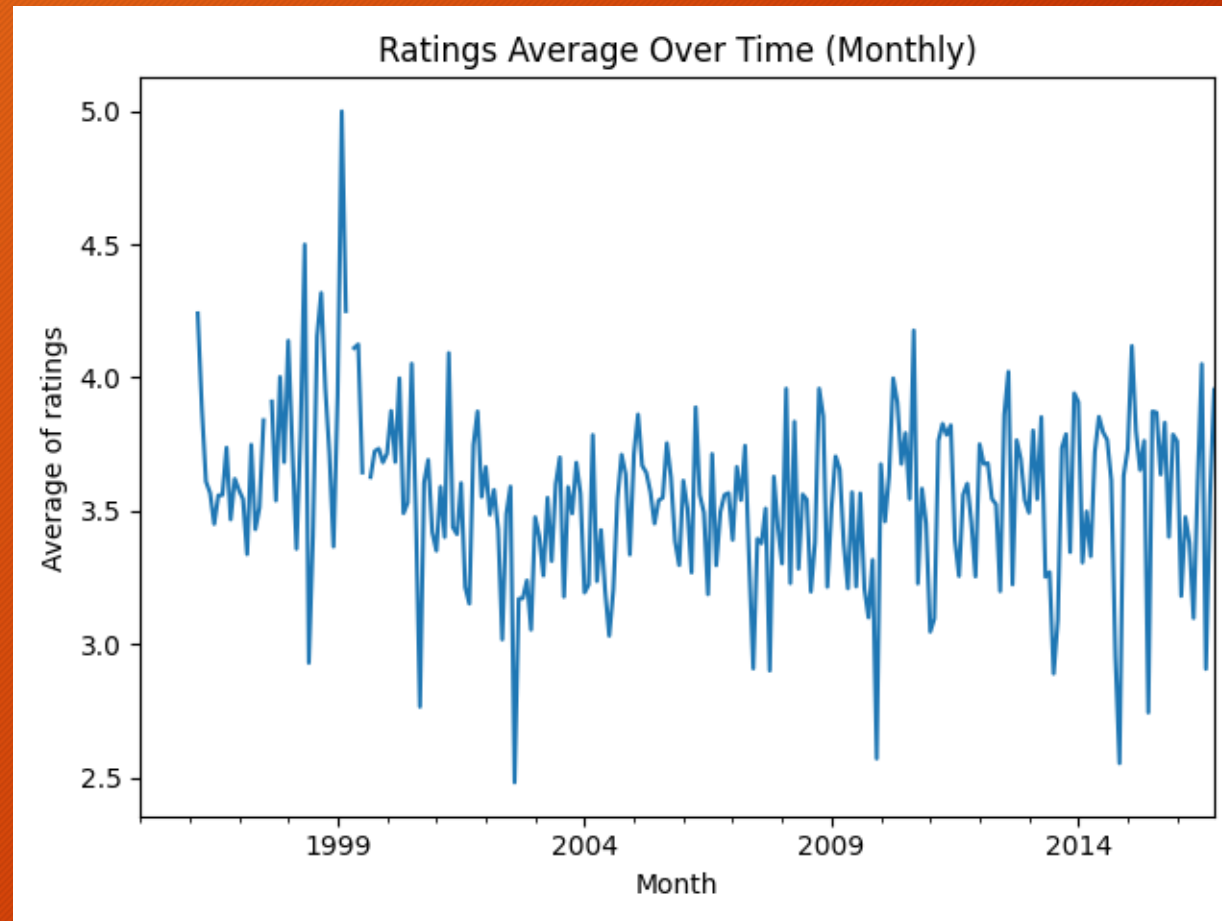
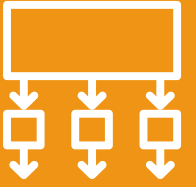
Dataset



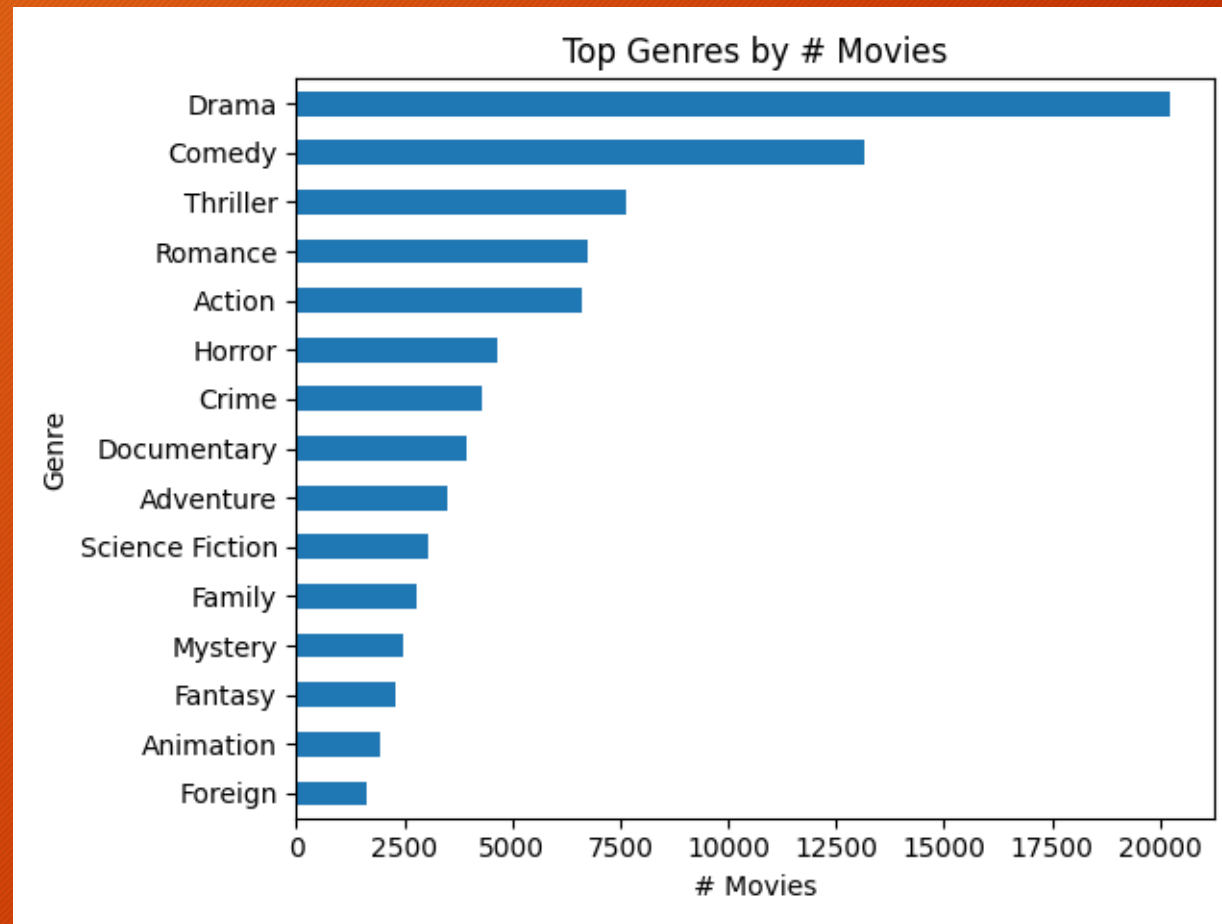
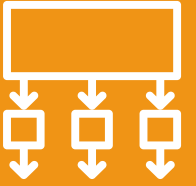
Dataset



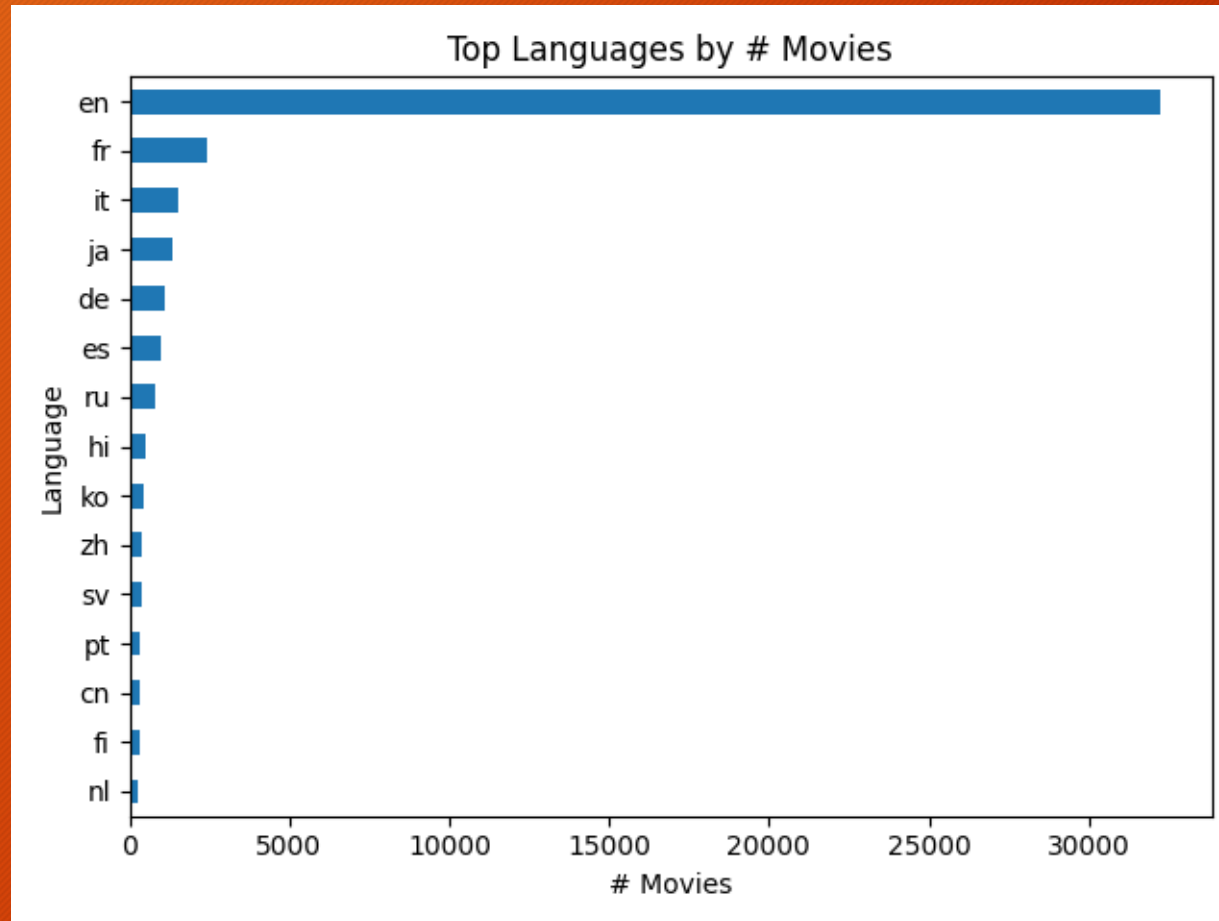
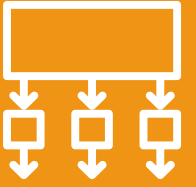
Dataset



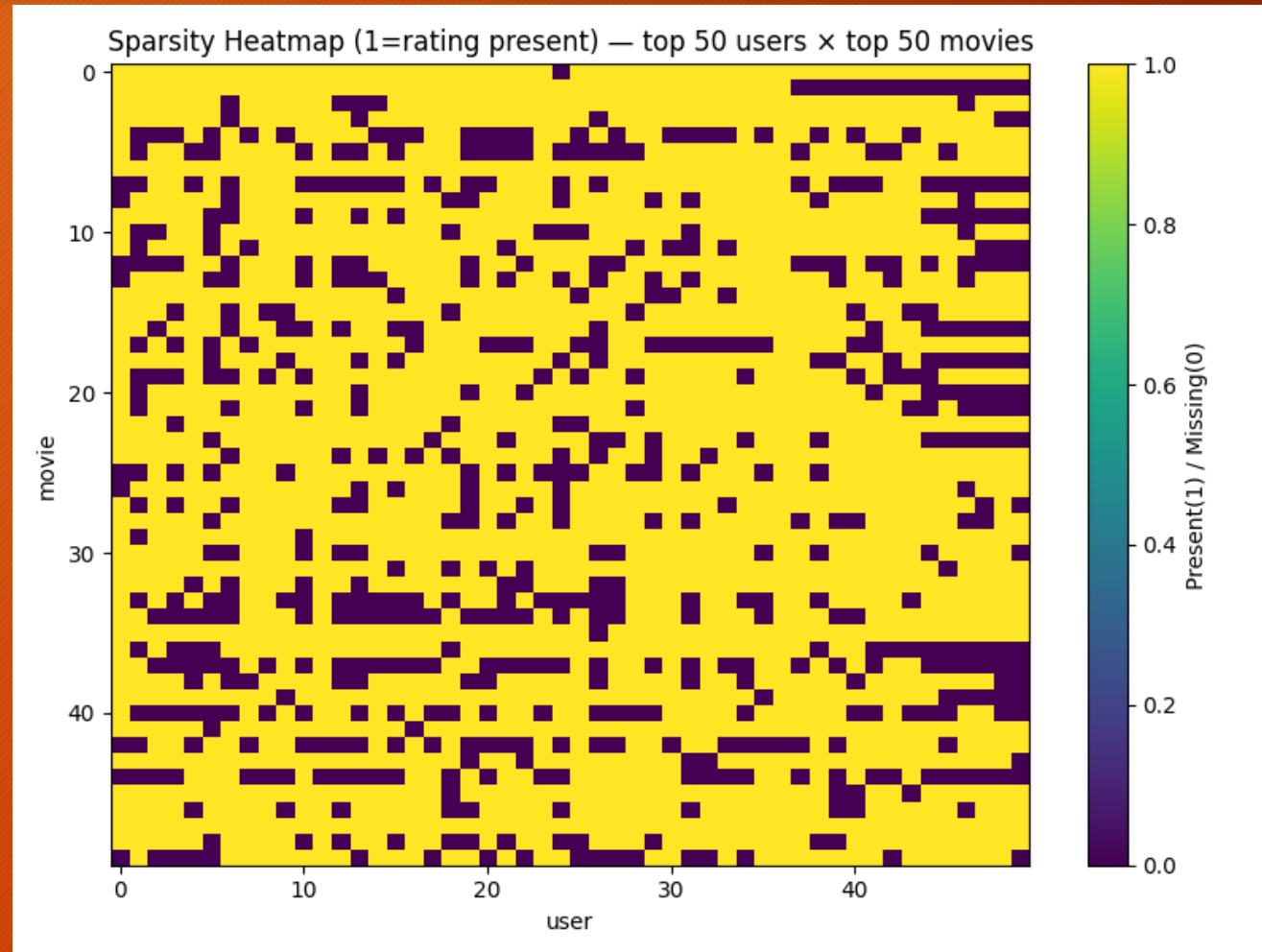
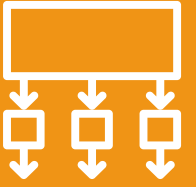
Dataset



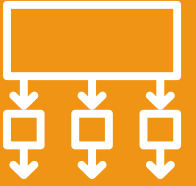
Dataset



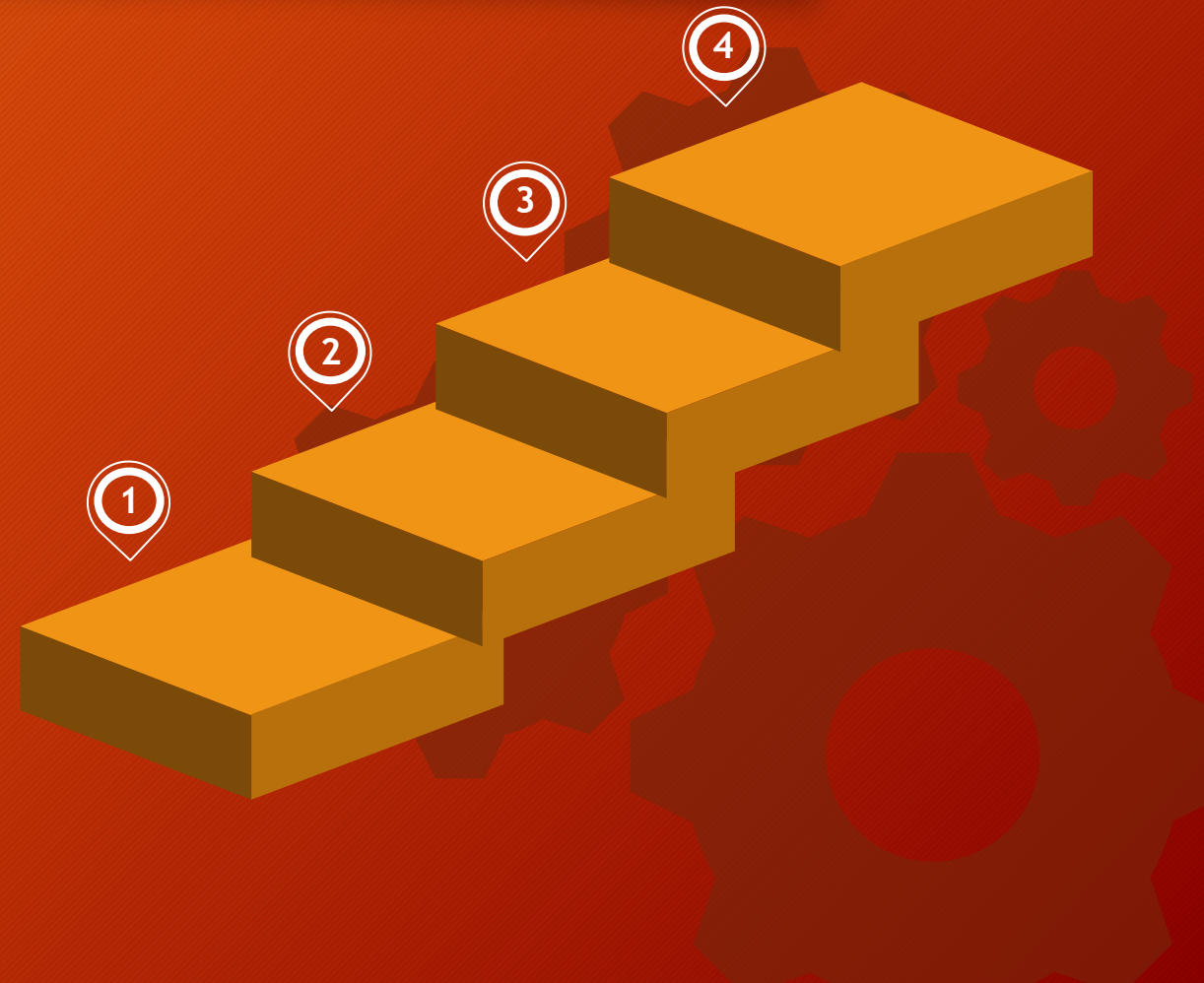
Dataset



Base Line(construction)



- r_{full} is grouped by tmdbId and title
- R_i : mean rating
- V_i : vote count
- Attach genres list

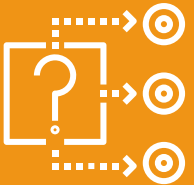


What does it return?

C: weighted mean of all ratings.

m: 80% of the vote counts.

WR: global weighted ratings.





We have two main approaches!

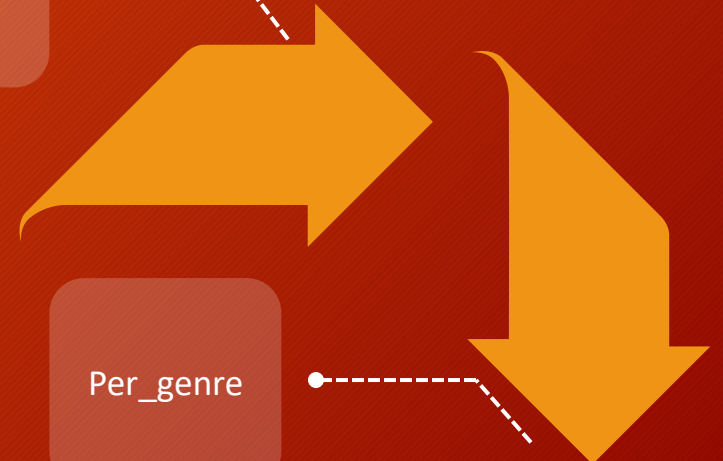
- Calculate weighted rating for a genre group.
- Calculating per-genre weighted ratings.

genre_wr(self, Cg):

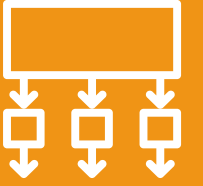
CG

Per_genre(self):

Per_genre



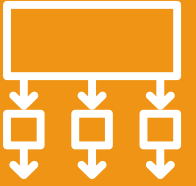
Global WR parameters:



- {C: np.float64(3.543), m: 13, m_quantile: 0.8}



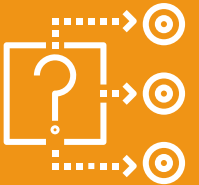
Top 10 globally popular movies (WR):



	title	vi	Ri	WR
200	The Shawshank Redemption	311	4.487138	4.449255
166	The Godfather	200	4.487500	4.429853
428	The Usual Suspects	201	4.370647	4.320368
168	The Godfather: Part II	135	4.385185	4.311208
288	Schindler's List	244	4.303279	4.264820
679	The Maltese Falcon	62	4.387097	4.240783
322	The African Queen	50	4.420000	4.239028
476	Pulp Fiction	324	4.256173	4.228661
572	Chinatown	76	4.335526	4.219761
372	Rear Window	92	4.315217	4.219607



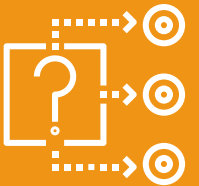
Final outputs



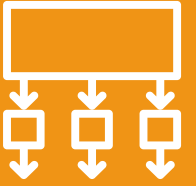
We have two csv files as below:

- `data/baseline/baseline_global_wr.csv`
- `data/baseline/baseline_per_genre_wr.csv`

Make a user profile and recommendations



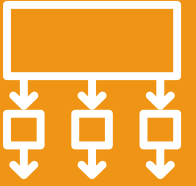
Make a user profile and recommendations



Text Features:

- Combines overview and tagline into a single text field.
- Applies TF-IDF vectorization with English stop words removal.

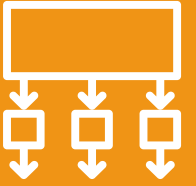
Make a user profile and recommendations



Text Features:

```
def setup(self):  
    # --- Text features (overview + tagline) ---  
    self.meta_subset["overview"] = self.meta_subset["overview"].fillna("")  
    self.meta_subset["tagline"] = self.meta_subset["tagline"].fillna("")  
    self.meta_subset["text_all"] = self.meta_subset["overview"] + " " + self.meta_subset["tagline"]  
    self.tfidf = TfidfVectorizer(stop_words="english", max_features=5000)  
    self.tfidf_matrix = self.tfidf.fit_transform(self.meta_subset["text_all"])
```

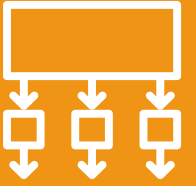
Make a user profile and recommendations



Categorical Features:

- **Genres:** Multi-label binarization of genre lists.
- **Keywords:** Multi-label binarization of keyword lists.
- **Cast & Crew:** Multi-label binarization of top cast and crew members.

Make a user profile and recommendations



Categorical Features:

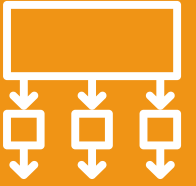
```
# --- Multi-hot keywords (if keywords dataset available) ---
# For now assume we have parsed keywords as a list in meta_clean["keywords_list"]
# If not, set as empty lists
if "keywords_list" not in self.meta_subset:
    self.meta_subset["keywords_list"] = [[] for _ in range(len(self.meta_subset))]
self.mlb_keywords = MultiLabelBinarizer()
self.keywords_matrix = self.mlb_keywords.fit_transform(self.meta_subset["keywords_list"])

# --- Top-k cast/crew multi-hot (if credits dataset available) ---
# Assume meta_subset["top_cast"] and ["top_crew"] are lists of names from preprocessing
if "top_cast" not in self.meta_subset:
    self.meta_subset["top_cast"] = [[] for _ in range(len(self.meta_subset))]
if "top_crew" not in self.meta_subset:
    self.meta_subset["top_crew"] = [[] for _ in range(len(self.meta_subset))]

self.mlb_cast = MultiLabelBinarizer()
self.cast_matrix = self.mlb_cast.fit_transform(self.meta_subset["top_cast"])

self.mlb_crew = MultiLabelBinarizer()
self.crew_matrix = self.mlb_crew.fit_transform(self.meta_subset["top_crew"])
```

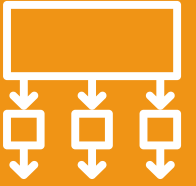
Make a user profile and recommendations



Feature Integration:

- Horizontally stacks all feature matrices using `hstack()`
- Creates comprehensive item feature representations.

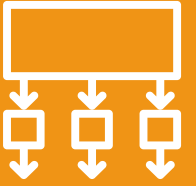
Make a user profile and recommendations



Feature Integration:

```
# --- Concatenate all features ---  
self.item_features = hstack([self.tfidf_matrix, self.genres_matrix, self.keywords_matrix, self.cast_matrix, self.crew_matrix])
```

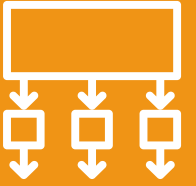
Make a user profile and recommendations



2. dimensionality_reduction()

- Applies Truncated SVD with 300 components for dimensionality reduction.
- Creates efficient item vectors indexed by tmdbId.
- Reduces computational complexity while preserving essential patterns.

Make a user profile and recommendations

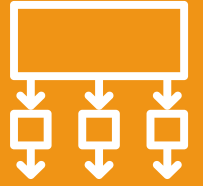


2. dimensionality_reduction()

```
1 usage  ─ Sajjad
def dimensionality_reduction(self):
    svd = TruncatedSVD(n_components=300, random_state=42)
    item_features_reduced = svd.fit_transform(self.item_features)
    # Map tmdbId → feature vector
    self.item_vectors = pd.DataFrame(item_features_reduced, index=self.meta_subset["id"])

    print("Recomendor dimensionality_reduction complete!")
```

Make a user profile and recommendations

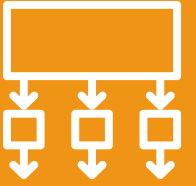


```
build_user_profile(user_id, ratings_df, item_vecs)
```

User Preference Modeling:

- Extracts the user's rating history.
- Applies mean-centering to adjust ratings.
- Creates a weighted average profile vector based on user preferences.
- Returns None for new users (cold start scenario)

Make a user profile and recommendations



```
1 usage  👤 Sajjad
def build_user_profile(self, user_id, ratings_df, item_vecs):
    # Get this user's ratings
    user_ratings = ratings_df[ratings_df["userId"] == user_id]
    if user_ratings.empty:
        return None

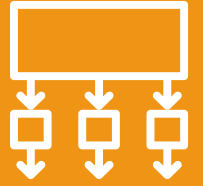
    # Mean center ratings
    mean_rating = user_ratings["rating"].mean()
    user_ratings = ratings_df[ratings_df["userId"] == user_id].copy()
    user_ratings["adj_rating"] = user_ratings["rating"] - mean_rating

    # Get feature vectors for rated items
    rated_vecs = item_vecs.loc[user_ratings["tmdbId"]]
    weights = user_ratings["adj_rating"].values.reshape(-1, 1)

    # Weighted average
    self.profile_vec = np.average(rated_vecs, axis=0, weights=weights.flatten())

    return self.profile_vec
```

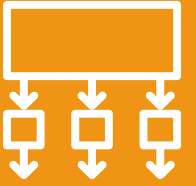
Make a user profile and recommendations



4. `recommend_content(user_id, ratings_df, item_vecs, top_n=10)`

- Builds user profile or uses global ratings for cold start
- Computes cosine similarity between user profile and all items
- Filters out previously rated items
- Returns top-N recommendations with titles and similarity scores

Make a user profile and recommendations



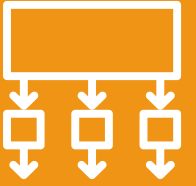
```
def recommend_content(self, user_id, ratings_df, item_vecs, top_n=10):
    profile = self.build_user_profile(user_id, ratings_df, item_vecs)
    if profile is None:
        # Cold start – fallback to global popularity
        return self.global_wr.head(top_n)[["title", "WR"]]

    # Compute cosine similarity to all items
    sims = cosine_similarity(X=[profile], item_vecs.values)[0]
    self.sim_df = pd.DataFrame({
        "tmdbId": item_vecs.index,
        "similarity": sims
    })

    # Exclude items already rated by the user
    seen = set(ratings_df.loc[ratings_df["userId"] == user_id, "tmdbId"])
    self.sim_df = self.sim_df[~self.sim_df["tmdbId"].isin(seen)]

    # Attach titles
    meta_titles = self.meta_subset.groupby("id")["title"].first()
    self.sim_df["title"] = self.sim_df["tmdbId"].map(meta_titles)
    return self.sim_df.sort_values(by="similarity", ascending=False).head(top_n)
```

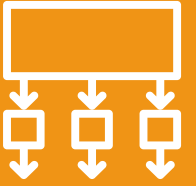
Make a user profile and recommendations



5. `explain_recommendation(tmdbId, user_id, ratings_df, meta_df)`

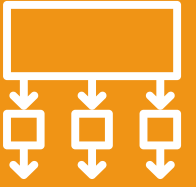
- Identifies genre and cast overlaps between recommended item and user's liked items.
- Generates human-readable explanations based on common features.
- Falls back to "matches your taste profile" when no obvious overlaps exist.

Make a user profile and recommendations



```
| def explain_recommendation(self, tmdbId, user_id, ratings_df, meta_df):  
    # Find top overlapping genres and cast with items user liked  
    rec_genres = set(meta_df.loc[meta_df["id"] == tmdbId, "genres"].values or [])  
    rec_cast = set(meta_df.loc[meta_df["id"] == tmdbId, "top_cast"].values or [])  
  
    # Get user's highly rated movies  
    liked = ratings_df[(ratings_df["userId"] == user_id) & (ratings_df["rating"] >= 4.0)]  
    liked_ids = liked["tmdbId"].tolist()  
  
    liked_genres = set()  
    liked_cast = set()  
    for mid in liked_ids:  
        liked_genres.update(meta_df.loc[meta_df["id"] == mid, "genres"].values or [])  
        liked_cast.update(meta_df.loc[meta_df["id"] == mid, "top_cast"].values or [])  
  
    genre_overlap = rec_genres & liked_genres  
    cast_overlap = rec_cast & liked_cast  
  
    self.explanation = []  
    if genre_overlap:  
        self.explanation.append(f"shares genres {'', '.join(sorted(genre_overlap))}")  
    if cast_overlap:  
        self.explanation.append(f"features cast members {'', '.join(sorted(cast_overlap))}")  
  
    return " and ".join(self.explanation) if self.explanation else "matches your taste profile"
```

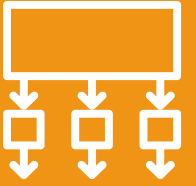
Make a user profile and recommendations



For example, I test this recommendation for ser 123!



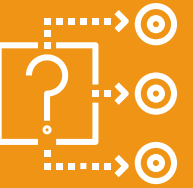
Make a user profile and recommendations



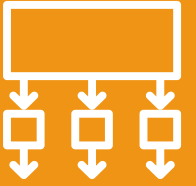
For example, I test this recommendation for ser 123!

```
Top 10 recommendations for user 123:  
The Bucks County Massacre - shares genres [{'id': 27, 'name': 'Horror'}]  
The Lost Honour of Christopher Jefferies - shares genres [{'id': 18, 'name': 'Drama'}]  
Fort Apache, the Bronx - matches your taste profile  
36th Precinct - matches your taste profile  
Naked Harbour - shares genres [{'id': 18, 'name': 'Drama'}]  
David Holzman's Diary - shares genres [{'id': 35, 'name': 'Comedy'}]  
Top Cat The Movie - matches your taste profile  
Babysitting - shares genres [{'id': 35, 'name': 'Comedy'}]  
Special Correspondents - shares genres [{'id': 35, 'name': 'Comedy'}]  
Evening's Civil Twilight in Empires of Tin - matches your taste profile
```

KNN recommendations



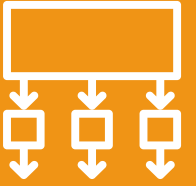
KNN recommendations



Key Components:

- Data Ingestion: The constructor takes a DataFrame `r_full` containing user-item interactions (columns: `userId`, `tmdbId`, `rating`).
- Index Mapping: Efficiently maps raw user and item IDs to internal matrix indices using pandas' category data type. This creates four dictionaries:
- `uid_map`, `iid_map`: Map internal index $\rightarrow$ original ID.
- `uid_inv`, `iid_inv`: Map original ID $\rightarrow$ internal index (crucial for fast lookups).

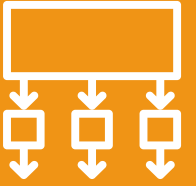
KNN recommendations



Key Components:

- **Sparse Matrix Construction:** Constructs a Compressed Sparse Row (csr_matrix) matrix R from the rating data. This is a memory-efficient representation for the user-item interaction matrix, especially since most users have not rated most items.

KNN recommendations



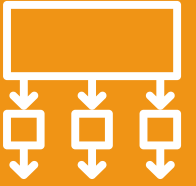
Key Components:

- tmdbId
- Rate

```
Sparse Matrix
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 99810 stored elements and shape (671, 9025)>

  Coords  Values
(0, 54)  4.0
(0, 99)  2.5
(0, 462) 2.0
(0, 534) 2.0
(0, 563) 3.0
(0, 588) 2.0
(0, 710) 4.0
(0, 721) 2.0
```

KNN Core Recommendation Algorithms



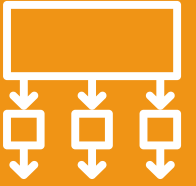
Item-Item KNN (item_item_knn)

Principle: "Users who liked this item also liked..."

Methodology:

- Identifies items a target user has already rated (seen_items).
- Computes the cosine similarity between every seen_item and every other item in the catalog.
- Generates a prediction score for each item by calculating a weighted average of the user's ratings, where the weights are the similarities.
- Masks items the user has already seen by setting their scores to $-\infty$, ensuring they are not recommended.

KNN Core Recommendation Algorithms



Item-Item KNN

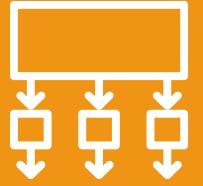
Top-10 Item-Item recommendations:

item_item_knn is called!

	tmbdId	score
0	56428	5.0
1	387893	5.0
2	86369	5.0
3	14317	5.0
4	14120	5.0
5	31094	5.0
6	12454	5.0
7	54795	5.0
8	52855	5.0
9	24012	5.0

The recommendations are often intuitive and justifiable.

KNN Core Recommendation Algorithms



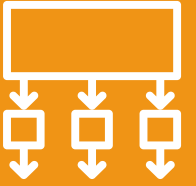
User-User KNN (user_user_knn)

Principle: "Users like you also liked..."

Methodology:

- Converts the sparse matrix to a dense array. (Note: This is a significant limitation for large datasets and should be replaced with sparse-aware computations for production use.)
- Mean-centers the user ratings to account for user bias (a harsh rater vs. a generous rater).
- Computes the cosine similarity between the target user's rating vector and all other users.
- Generates a prediction score by taking a weighted average of the ratings from the most similar users (neighbors).
- Adds the target user's mean rating back to the centered predictions.
- Masks already seen items.

KNN Core Recommendation Algorithms



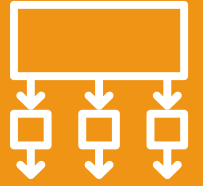
User-User KNN (user_user_knn)

```
Top-10 User-User recommendations:  
user_user_knn is called!
```

	tmdbId	score
0	14	4.021069
1	680	3.945359
2	424	3.935967
3	278	3.910818
4	2493	3.908165
5	275	3.904795
6	510	3.902942
7	9377	3.890176
8	85	3.883837
9	104	3.876588

Directly leverages the wisdom of a user's nearest neighbors.

KNN Core Recommendation Algorithms



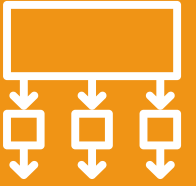
Matrix Factorization (train_mf and mf_scores)

Principle: Discovers latent features (e.g., genre, director, themes) that describe users and items.

Methodology (Biased SGD):

- Model: The rating prediction is decomposed into a global mean (μ), user bias (b_u), item bias (b_i), and the dot product of user and item latent factor vectors (P and Q).
- Training: Uses Stochastic Gradient Descent (SGD) to minimize the prediction error on known ratings. The algorithm iteratively updates the biases and latent factors over multiple epochs.
- Prediction: After training, scores for all items for a user are calculated using the learned parameters.
- Strengths: Generally offers higher accuracy and can capture complex, non-linear relationships that KNN methods might miss. It is more scalable to very large datasets.

KNN Core Recommendation Algorithms



Matrix Factorization (train_mf and mf_scores)

Generally offers higher accuracy and can capture complex, non-linear relationships that KNN methods might miss. It is more scalable to very large datasets.

Top-10 MF recommendations:

	tmdbId	score
0	4912	6.741857
1	5511	5.632943
2	278	4.863644
3	121	4.858496
4	762	4.772880
5	11645	4.759062
6	265189	4.752412
7	654	4.749147
8	2493	4.748598
9	3078	4.739947

Thanks for your attention

